Python Twitter Bot for Tweeting and Retweeting

Introduction:

This project implements a Twitter bot using the **Tweepy** library in **Python**. The goal of our project is to automate the basic social media activities of Twitter — like **tweeting** and **retweeting** — while maintaining realistic human-like behaviour using "**random and time module**". The bot interacts with the **Twitter API v2** through secure authentication and logs these actions in a log file for after-run viewing. Sensitive authentication data like **API keys**, **API tokens**, and **Bearer ID** are handled securely through the "**os module"** and environment variables, ensuring that credentials are never exposed in the source code.

Methodology:

The project integrates several Python modules to achieve a robust, realistic automation flow:

1. **Authentication:**
   The bot loads credentials from environment variables (.env) for the Twitter API. It uses Tweepy's OAuth 1.0a user context for authorization and API interaction.

2. **Action Logic:**
   This program defines two major operations — tweeting random content and retweeting random posts from a predefined list of phrases and posts ID's, the bot randomly selects entries using the **random** module.

3. **Rate-Limit Handling and Randomization:**
   Randomized waiting intervals are added between actions using "random_delay()" function with **time** module. This functions simulates natural user activity to make actions look natural and avoid rate-limit issues. Additional logic ensures variation between short, medium, and long delays, producing unpredictable posting intervals.

4. **Logging and Monitoring:**
   The **logging** module records all events — including tweets, retweets, delays, and errors — with timestamps. Logs are saved to a file (bot.log) for post-run review.

5. **Error Handling:**
   The bot handles exceptions such as network errors, invalid tweet IDs, and rate-limit violations (HTTP 429). When a rate limit is hit, the bot pauses for an extended duration before retrying.

Applications:

This project demonstrates how Python can be used safely & efficiently to interact with social media APIs for legitimate automation tasks. Potential applications include:

- **Content Scheduling:** Automating promotional or informational tweets at random or predefined intervals.

- **Engagement Boosting:** Retweeting and interacting with selected accounts to maintain consistent visibility.

- **Educational Use:** Demonstrating secure API usage, environment variable handling, and ethical automation in software engineering or data science courses.

Conclusion: The project provides a overall practical, modular foundation for social media automation that emphasizes **security, reliability, and responsible design** within the limitations of Twitter's free API tier.
Github Link: https://github.com/PV-Shreyan/PWP-Project.git